

CHAPTER-1

INTRODUCTION

Voice Assistant to provide an efficient and user-friendly experience for a wide range of users. With this voice assistant, you can easily access information, perform tasks, and receive personalized recommendations using voice commands.

Our voice assistant is powered by advanced natural language processing (NLP) technology, which allows it to understand and interpret spoken commands and questions. It is designed to be accessible through various devices, such as smartphones, smart speakers, and laptops, making it easy for anyone to use. Throughout this documentation, we will provide an overview of the voice assistant's features and functionalities, as well as the technologies and tools we used to develop it. We will also discuss the challenges we faced during the development process and how we overcame them.

We hope this documentation will be helpful in understanding the capabilities of our voice assistant and how it can benefit you in your daily life. Thank you for your interest in our project!

1.1 SCOPE

The scope of Python-based voice assistants is vast, encompassing various technological, developmental, and application aspects. These assistants leverage Python libraries for speech recognition, natural language processing (NLP), and text-to-speech (TTS) conversion, enabling seamless human-computer interaction. They find applications in personal assistance, customer service, home automation, and accessibility tools, enhancing user experiences across different domains. Additionally, Python-based voice assistants serve as educational tools for learning about AI and machine learning, provide business opportunities in developing innovative products, and offer a platform for advanced research in NLP and human-computer interaction. As technology evolves, these voice assistants are poised to become even more integral to everyday life and industry.

1.2 OBJECTIVE

The objective of this Python-based voice assignment project is to develop a comprehensive voice-controlled application that leverages natural language processing (NLP) and speech recognition technologies to create an intuitive and interactive user experience. The project aims to integrate Python with various APIs and libraries such as SpeechRecognition, Pyttsx3, and NLP frameworks to facilitate seamless voice interaction. The core focus is on enabling users to perform a wide range of tasks, such as setting reminders, searching the web, controlling smart home devices, and managing daily schedules, through voice commands. By emphasizing accuracy, responsiveness, and user-friendly design, this project seeks to enhance accessibility and convenience for users, particularly those with physical disabilities or those who prefer hands-free operation. Additionally, the project will explore advanced features such as voice authentication and personalization to ensure security and a tailored user experience. Ultimately, the goal is to create a robust and versatile voice assistant that can be easily adapted and expanded for various applications and user needs.

1.3 PURPOSE

The purpose of developing Python-based voice assistants is to create intelligent systems that facilitate natural and efficient human-computer interactions. These assistants aim to enhance user convenience by performing tasks through voice commands, such as setting reminders, answering queries, controlling smart home devices, and providing real-time information. They also serve to improve accessibility for individuals with disabilities, enabling hands-free operation and access to digital resources. Furthermore, Python-based voice assistants provide valuable learning experiences for developers and researchers, offering insights into speech processing, natural language understanding, and artificial intelligence. Ultimately, their purpose is to integrate seamlessly into various aspects of daily life and industry, improving productivity, user engagement, and overall technological interaction.

CHAPTER-2

LITERATURE SURVEY

In today's world, machines are trained to think like humans and perform tasks on their own, replacing what humans can do. Based on this situation, the concept of voice assistants, which can perform various human tasks based on the human voice, was born. A virtual assistant can filter the voice commands given by the user and return relevant information. People around the world are transforming their digital experiences with future technologies such as virtual reality, augmented reality, and voice interaction. Voice assistants are emerging as a new evolution in human-machine interaction, where analog signals are converted into digital waves by audio signals. Over the past few years, smartphone usage has grown significantly, leading to the widespread use of voice assistants such as Apple's Siri, Google Assistant, Microsoft's Cortana, and Amazon's Alexa. Voice assistants are built using technologies such as speech recognition, text-to-speech, and natural language processing (NLP) that offer unlimited applications to make users' lives easier and more convenient.

According to Deepak Shende, Ria Umabiya, AIVA (Microsoft, Google's Google Assistant and the smartest assistant named "AIVA" 2018) aims to create a voice assistant that can do many things, such as searching the Internet. It has new features such as commenting on social media such as Facebook and Twitter. With just few simple commands you can learn about the weather around you and get information about the weather in your area.

Tulshan explains that user's fingers can be injured due to constant typing. In order to avoid such problems, we need to create a system that allows us to do everything with voice commands. The speech will be recognized by the system. The recognition words will be compiled, if necessary are clarified, then printed on the screen, and after this again the recognized word will be matched with a specific keyword and if the match is found, then program will be compiled and executed.

CHAPTER-3

SYSTEM ANALYSIS

System analysis for Python-based voice assistants involves a detailed examination of the components, functionalities, and interactions within the system. This process aims to identify the system's requirements, design, and implementation challenges, ensuring the development of a robust and efficient voice assistant.

3.1 SOFTWARE REQUIREMENT SPECIFICATIONS

The software requirement specifications (SRS) for a Python-based voice assistant encompass a detailed examination of both functional and non-functional requirements. The primary functional requirements include the capability for speech recognition, natural language processing (NLP), text-to-speech (TTS) conversion, and command execution. Using Python libraries such as SpeechRecognition and PyAudio, the system will convert spoken language into text, handle diverse accents, and manage background noise. NLP will be implemented using libraries like NLTK, spaCy, and Transformers to understand user intent and support multiple languages. The TTS functionality, provided by pyttsx3 and GTTS, will convert textual responses into natural and clear speech. Additionally, the system will execute various commands such as setting reminders, fetching information, and controlling smart devices, ensuring accurate task execution and providing meaningful feedback to users. Error handling is also a crucial aspect, with the system designed to manage errors gracefully and inform users appropriately.

3.1.1 Functional Requirements:

- ✓ **Speech Recognition:** The system must accurately convert spoken language into text.
- ✓ **Natural Language Understanding:** The system should understand and process the text to determine user intent.
- ✓ **Text-to-Speech (TTS):** The system must convert text responses back into speech.

- ✓ **Command Execution:** The system should execute various commands like setting reminders, fetching information, and controlling devices.
- ✓ **Error Handling:** The system must handle errors gracefully and provide meaningful feedback to users.

3.1.2 Non-Functional Requirements:

- ✓ **Performance:** The system should respond quickly and handle multiple requests efficiently.
- ✓ **Scalability:** The system should support the addition of new functionalities and integration with other services.
- ✓ **Usability:** The system must provide a user-friendly interface and clear interactions.
- ✓ **Security:** The system should ensure the privacy and security of user data.
- ✓ **Reliability:** The system must be reliable and function correctly under various conditions.

3.2 SYSTEM REQUIREMENTS

3.2.1 HARDWARE REQUIREMENTS:

- ✓ Operating System : Windows 10
- ✓ Processor : Intel® Core(TM) i5-3470 CPU
- ✓ RAM : 8 GB
- ✓ System type : 64-bit operating system

3.2.2 SOFTWARE REQUIREMENTS:

- ✓ Python 3.12 or
- ✓ JavaScript
- ✓ Windows 10 or above
- ✓ Chrome Driver

CHAPTER-4

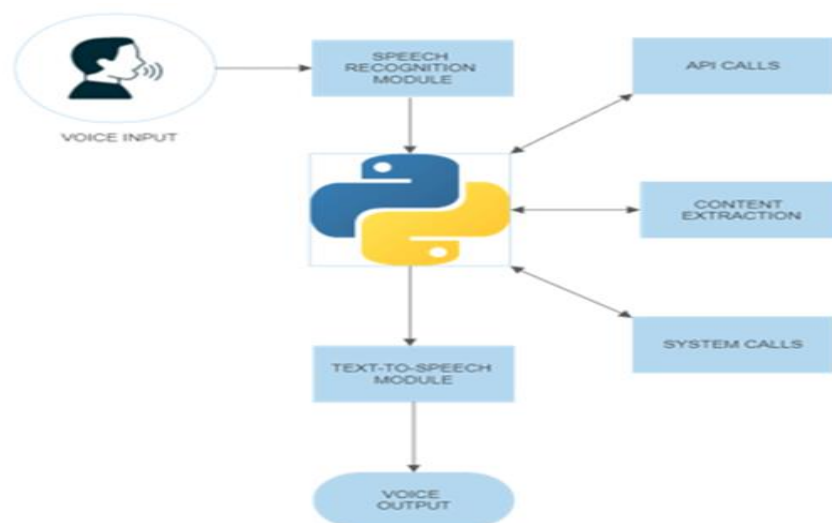
SYSTEM DESIGN

The system design for a Python-based voice assistant involves structuring the application's architecture, defining its components, and illustrating the flow of data and interactions. This design aims to meet the software requirements specified in the earlier phase and ensure the efficient implementation of core functionalities like speech recognition, natural language processing (NLP), text-to-speech (TTS) conversion, and command execution.

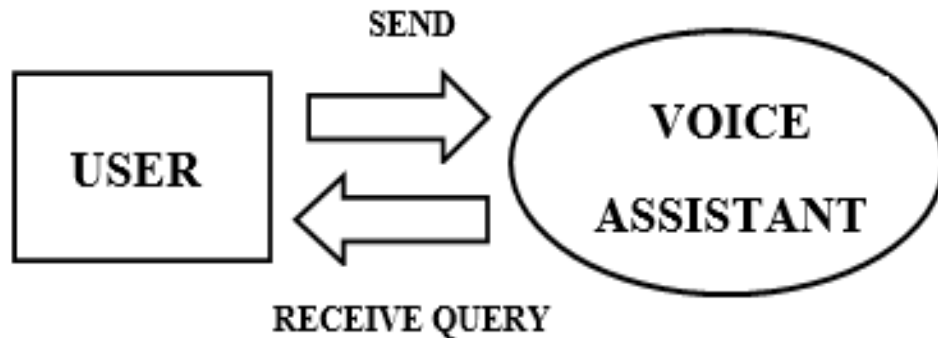
4.1 SYSTEM ARCHITECTURE

The voice assistant system will be designed using a layered architecture approach to promote modularity, maintainability, and scalability:

- ✓ **Presentation Layer:** This layer manages the user interface, where users interact with the voice assistant through spoken commands and receive auditory feedback.
- ✓ **Application Layer:** This layer contains the business logic of the voice assistant, including modules for speech recognition, NLP, command execution, and TTS. Each module interfaces with external APIs and libraries such as SpeechRecognition, NLTK, spaCy, pyttsx3, and gTTS.
- ✓ **Data Layer:** Although minimal, this layer manages any persistent data storage requirements, such as user preferences or session information.



4.2 DATAFLOW DIAGRAM



4.3 MODULES

pyttsx3

- ✓ **Description:** pyttsx3 is a Python library that converts text to speech (TTS). It works offline and is compatible with both Python 2 and 3, making it a versatile choice for applications requiring speech synthesis without an internet connection.
- ✓ **Key Features:**
 - Supports multiple speech engines: SAPI5 on Windows, NSSpeechSynthesizer on MacOS, and espeak on Linux.
 - Allows adjustment of speech rate, volume, and voice.
 - Can save speech to an audio file.

speech_recognition

- ✓ **Description:** The speech_recognition library provides a way to convert spoken language into text using various speech recognition engines and APIs. It supports a range of services including Google Web Speech API, Microsoft Bing Voice Recognition, and more.
- ✓ **Key Features:**
 - Captures audio from the microphone or audio files.
 - Interfaces with several online and offline speech recognition engines.

- Offers easy-to-use methods for recognizing speech and handling errors.

Eel

- ✓ **Description:** Eel is a Python library for creating simple desktop applications with HTML, CSS, and JavaScript. It bridges the gap between web and desktop development by allowing Python and JavaScript to communicate seamlessly.
- ✓ **Key Features:**
 - Enables building cross-platform desktop applications using web technologies.
 - Supports bidirectional communication between Python and JavaScript.
 - Simple API for exposing Python functions to JavaScript and vice versa.

webbrowser

- ✓ **Description:** The webbrowser module in Python provides a high-level interface to display web-based documents to users. It can open URLs in the default web browser or a specified browser.
- ✓ **Key Features:**
 - Opens web pages in a web browser.
 - Allows specifying which browser to use.
 - Supports opening new browser windows or tabs.

time

- ✓ **Description:** The time module provides various functions related to time. It includes functions for getting the current time, measuring elapsed time, and sleeping for a specified number of seconds.
- ✓ **Key Features:**
 - Functions for manipulating time representations.

- Sleep function for pausing execution.
- Performance measurement functions.

pyautogui

- ✓ **Description:** PyAutoGUI is a Python module that provides a way to programmatically control the mouse and keyboard. It is useful for automating tasks on the computer, such as clicking, typing, and taking screenshots.
- ✓ **Key Features:**
 - Simulates mouse movements, clicks, and drags.
 - Sends keyboard input, including typing and hotkeys.
 - Captures screenshots and locates images on the screen.

subprocess

- ✓ **Description:** The subprocess module allows you to spawn new processes, connect to their input/output/error pipes, and obtain their return codes. It provides a powerful interface for running and interacting with system commands.
- ✓ **Key Features:**
 - Spawns and manages subprocesses.
 - Provides methods for piping data between processes.
 - Allows capturing and handling output and errors.

playsound

- ✓ **Description:** playsound is a simple Python module for playing sound files. It supports WAV and MP3 formats and requires only the path to the sound file to play it.
- ✓ **Key Features:**
 - Plays sound files with a single function call.
 - Supports WAV and MP3 files.
 - Works on multiple platforms (Windows, MacOS, and Linux).

hugchat

- ✓ **Description:** HugChat is a client library for interacting with Hugging Face's chat models. It provides an easy-to-use interface for generating responses from chat models hosted on Hugging Face.
- ✓ **Key Features:**
 - Simplifies interaction with Hugging Face's chat models.
 - Allows customization of requests.
 - Handles responses from chat models effectively.

Particle.js

- ✓ **Description:** Particle.js is a lightweight JavaScript library for creating particles and particle effects. It is often used for creating interactive and visually appealing backgrounds for websites.
- ✓ **Key Features:**
 - Allows you to create particles of various shapes, sizes, and colors.
 - Provides options for interactive effects, such as particles reacting to mouse movement or clicks.
 - Highly customizable with options for particle count, size, speed, and more.

Textillate.js

- ✓ **Description:** Textillate.js is a JavaScript library for animating text. It combines the power of animate.css and lettering.js to provide a simple interface for applying a wide range of text animations.
- ✓ **Key Features:**
 - Allows you to apply a variety of animations to text, such as fading, bouncing, rotating, and more.
 - Supports sequencing animations, making it easy to create complex animation sequences.

CHAPTER-5

IMPLEMENTATION

5.1 TECHNOLOGIES

PYTHON

Python has a lot of libraries for every need of this project. For this project, we used libraries such as `speech_recognition` to recognize voice and `pyttsx3` for text-to-speech conversion. Python is reasonably efficient. Efficiency is usually not a problem for small examples. If your Python code is not efficient enough, a general procedure to improve it is to identify the bottlenecks, profile the code to determine what is taking most of the time, and then implement just those critical parts more efficiently in some lower-level language like C or C++. This will result in much less programming effort and more efficient code because you will have more time to optimize than writing everything from scratch in a low-level language. Additionally, Python's integration with tools like Cython or PyPy can also help boost performance while maintaining the simplicity of Python.

Python is a computer programming language often used to build websites and software, automate tasks, and conduct data analysis. It is a general-purpose language, meaning it can be used to create a variety of different programs and isn't specialized for any specific problems. Its simplicity and readability make it an excellent choice for beginners, while its power and versatility are appreciated by experienced developers. Python's extensive standard library and supportive community contribute to its efficiency and flexibility. This language is highly regarded in fields such as machine learning, artificial intelligence, scientific computing, and web development.

Python supports various programming paradigms like structured programming, functional programming, and object-oriented programming. However, the most important fact is that the Object-Oriented approach of Python allows its users to implement the concepts of Encapsulation, Inheritance, Polymorphism, etc. Python's dynamic typing and memory management features make it suitable for rapid application development and scripting.

JAVA SCRIPT

Integrating a Python-based voice assistant with JavaScript can create a seamless and interactive user experience by combining the strengths of both languages. Python, with its robust libraries like ``speech_recognition`` for speech-to-text conversion and ``pyttsx3`` for text-to-speech synthesis, forms the backbone of the voice assistant, handling the core functionalities of voice input and output. These capabilities can be further enhanced by using other Python libraries such as ``pyautogui`` for automating GUI interactions or ``subprocess`` for executing system commands. Once the voice assistant processes the voice input and generates the corresponding response, JavaScript comes into play on the front-end to manage the user interface and provide real-time interactivity. For example, JavaScript can be used to dynamically update the web page content based on the voice assistant's responses, create engaging animations using libraries like ``Textillate.js``, or even control visual effects with ``Particle.js``. By leveraging JavaScript's ability to manipulate the DOM, the user interface can be made highly responsive, ensuring that voice interactions translate into immediate visual feedback.

To establish communication between the Python backend and the JavaScript frontend, various techniques can be employed. One common approach is using a web framework like Flask or Django in Python to create a RESTful API. The voice assistant can process the voice commands and send the data to the frontend via HTTP requests. JavaScript, on the client side, can make AJAX calls to these API endpoints, retrieve the processed data, and update the web interface accordingly. For more real-time interactions, WebSockets can be utilized to maintain a persistent connection between the client and server, allowing instantaneous data exchange. This method is particularly useful for applications requiring continuous updates or streaming data. Additionally, using a library like Eel can simplify the process of linking Python and JavaScript by allowing Python functions to be exposed to JavaScript directly, facilitating a smoother integration. By combining Python's powerful data processing capabilities with JavaScript's dynamic user interface management, developers can create sophisticated and responsive voice-assisted applications that offer a rich user experience.

5.2 SAMPLE CODE

main.py

```
import eel

import os

from engine.features import *

from engine.command import *

eel.init("www")

play_sound()

os.system('start msedge.exe --app="http://localhost:8000/index.html"')

eel.start('index.html',mode=None, host="localhost", block=True)
```

index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>CORAL</title>

    <link rel="shortcut icon" href="bot_files/images/IMAGE.png" type="image/x-icon">

    <!-- Bootstrap -->

    <link      href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet"
```

```

    integrity="sha384-
EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTWfSpd3yD65VohhpuuCOmLASjC"
crossorigin="anonymous">

<!-- Bootstrap Icons -->

<link          rel="stylesheet"          href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.11.1/font/bootstrap-icons.css">

<!-- Particle js -->

<script src="https://cdnjs.cloudflare.com/ajax/libs/modernizr/2.8.3/modernizr.min.js"

    type="text/javascript"></script>

<!-- Texllate -->

<link rel="stylesheet" href="vendore\textillate\animate.css">

<link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">

    <section id="Oval-Section" class="mb-4">

        <div class="row">

            <div class="col-md-1"></div>

            <div class="col-md-10">

                <div class="d-flex justify-content-center align-items-center" style="height:
80vh;">

                    <canvas id="canvasOne" width="700" height="420" style="position:
absolute;"></canvas>

                    <div id="edith">

```

```

    <div class="square">

        <span class="circle"></span>

        <span class="circle"></span>

        <span class="circle"></span>

    </div>

</div>

</div>

<h5 class="text-center text-light heading" style="font-size: 30px;">Hello ,I am
Coral</h5>

<div class="col-md-12 mt-4 pt-4">

    <div class="text-center micbtn">

        <button class="btn glow-on-hover micbutton" id="micbutton"
onclick="startListening()">

            <i class="bi bi-mic"></i>

        </button>

    </div>

</div>

</div>

<div class="col-md-1"></div>

</div>

</section>

<section id="SiriWave-Section" class="mb-4">

    <div class="container">

```

```

<div class="row">

  <div class="col-md-12">

    <div class="d-flex flex-column justify-content-center align-items-center"
      style="height: 100vh;">

      <div class="">

        <p class=" text-start mb-4 siri-msg edith" style="font-size:
28px;">Hello, I am Coral</p>

        <div id="siri-container">

          </div>

        </div>

      </div>

    </div>

  </div>

</div>

</section>

</div>

<!--Jquery -->

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.4/jquery.min.js"></script>

<!-- Bootstrap -->

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"

```



```

        integrity="sha384-
MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM
"

        crossorigin="anonymous"></script>

<!-- Particle js -->

<script src="https://cdnjs.cloudflare.com/ajax/libs/jquery/2.1.3/jquery.min.js"></script>

<script          src="https://ajax.googleapis.com/ajax/libs/jqueryui/1.11.2/jquery-
ui.min.js"></script>

<script src="script.js"></script>


<!-- Siri wave -->

<script src="https://unpkg.com/siriwave/dist/siriwave.umd.min.js"></script>

<!-- Texllate js -->

<script src="vendore\texillate\jquery.fittext.js"></script>

<script src="vendore\texillate\jquery.lettering.js"></script>

<script src="http://jschr.github.io/textillate/jquery.textillate.js"></script>

<script src="main.js"></script>

<script src="controller.js"></script>

<script src="/eel.js"></script>

</body>

</html>

```

style.css

```
body{

    background-color: rgb(7, 7, 7);

    overflow-y: hidden;

    overflow-x: hidden;

}

.square{

    position: relative;

    width: 400px;

    height: 400px;

    display: flex;

    justify-content: center;

    align-items: center;

}

.square span:nth-child(1){

    position: absolute;

    top: 0;

    left: 0;

    width: 100%;

    height: 100%;

    background-image: radial-gradient(#6b72ff00 50%, #18c36d3b 40%);

    box-shadow: 0 0 50px rgb(43, 209, 65), inset 0 0 50px rgb(27, 175, 71);
```

```

border-radius: 38% 62% 63% 37% / 41% 44% 56% 59%;

transition: 0.5s;

animation: animate1 6s linear infinite;
}

.square span:nth-child(2){

    position: absolute;

    top: 0;

    left: 0;

    width: 100%;

    height: 100%;

    background-image: radial-gradient(#6b72ff00 50%, #2894613b 40%);

    box-shadow: 0 0 50px rgb(53, 223, 180), inset 0 0 50px rgb(32, 183, 32);

    border-radius: 38% 62% 63% 37% / 41% 44% 56% 59%;

    transition: 0.5s;

    animation: animate2 4s linear infinite;
}

.square span:nth-child(3){

    position: absolute;

    top: 0;

    left: 0;

    width: 100%;

    height: 100%;

```

```

background-image: radial-gradient(#1fa69600 50%, #244b313b 40%);

box-shadow: 0 0 50px rgb(0, 255, 72), inset 0 0 50px rgb(42, 239, 95);

border-radius: 38% 62% 63% 37% / 41% 44% 56% 59%;

transition: 0.5s;

animation: animate3 8s linear infinite;

}

@keyframes animate1 {

    0%{

        transform: rotate(0deg);

    }

    100%{

        transform: rotate(360deg);

    }

}

@keyframes animate2 {

    0%{

        transform: rotate(0deg);

    }

    100%{

        transform: rotate(360deg);

    }

}

```

```

@keyframes animate3 {

    0%{

        transform: rotate(0deg);

    }

    100%{

        transform: rotate(360deg);

    }

}

```

```

/* Input Filed Start*/

```

```

#TextInput {

    background-color: #181818a8;

    border-color: blue;

    box-shadow: 0 0 20px rgb(9, 255, 0),

        inset 0 0 0px rgb(25, 0, 255);

    border-radius: 50p;

    color: white;

    padding: 3px 0px 3px 20px;

    margin: 0px 20%;

}

```

```
.input-field {  
  
  background-color: transparent;  
  
  border: none;  
  
  width: 95%;  
  
  outline: none;  
  
  color: white;  
  
  font-family: cursive;  
  
}
```

```
.glow-on-hover {  
  
  width: 70px;  
  
  height: 70px;  
  
  
  
  outline: none;  
  
  color: #fff;  
  
  background: #111;  
  
  cursor: pointer;  
  
  position: relative;  
  
  z-index: 0;  
  
  border-radius: 10px;  
  
  padding: 0px;  
  
  margin-left: 10px;  
  
}
```

```
.glow-on-hover:before {  
  
    content: "";  
  
    background: linear-gradient(45deg, #ff0000, #ff7300, #ffff00, #48ff00, #00ffd5,  
#002bff, #7a00ff, #ff00c8, #ff0000);  
  
    position: absolute;  
  
    top: -2px;  
  
    left: -2px;  
  
    background-size: 400%;  
  
    z-index: -1;  
  
    filter: blur(5px);  
  
    width: calc(100% + 4px);  
  
    height: calc(100% + 4px);  
  
    animation: glowing 20s linear infinite;  
  
    opacity: 0;  
  
    transition: opacity .3s ease-in-out;  
  
    border-radius: 10px;  
  
}  
  
.glow-on-hover:active {  
  
    color: #181818a8  
  
}
```

```
.glow-on-hover:active:after {  
  
    background: transparent;  
  
}
```

```
.glow-on-hover:hover:before {  
  
    opacity: 1;  
  
}
```

```
.glow-on-hover:after {  
  
    z-index: -1;  
  
    content: "";  
  
    position: absolute;  
  
    width: 100%;  
  
    height: 100%;  
  
    background: #111;  
  
    left: 0;  
  
    top: 0;  
  
    border-radius: 10px;  
  
}
```

```
@keyframes glowing {  
  
    0% {
```



```
background-position: 0 0;

}

50% {

background-position: 400% 0;

}

100% {

background-position: 0 0;

}

}

/* Input Filed End*/

/* Chat Box Start */

.chat-canvas{

background-color: #191919

}

.receiver_message{
```

```
padding: 8px;

border: 2px solid cyan;

border-radius: 0px 15px 15px 20px;

width: auto;

color: white;

background-color: #0dcaf014;

}
```

```
.sender_message{

padding: 8px;

border: 1px solid #0045ff;

border-radius: 15px 15px 0px 20px;

width: auto;

color: white;

background-color: #0045ff;

}
```

```
.width-size{

max-width: 80%;

width: auto;

}
```

```
/* Chat Box Start End*/
```

```
.sirimsg{  
  
  font-size: clamp(28px);  
  
}
```

```
.edith{  
  
  color: rgb(241, 237, 237);  
  
}
```

```
.micbutton{  
  
  width: 70px;  
  
  height:60px;  
  
  border-radius:50px;  
  
}
```

main.js

```

$(document).ready(function () {

    let i=0;

    $('.heading').textillate({

        loop: true,

        sync: true,

        in: {

            effect: "bounceIn",

        },

        out: {

            effect: "bounceOut",

        },

    });

    // Siri configuration

    var siriWave = new SiriWave({

        container: document.getElementById("siri-container"),

        width: 800,

        height: 200,

        style: "ios9",

        amplitude: "1",

        speed: "0.30",

        autostart: true
    });

```

```

});

// Siri message animation

$('.siri-msg').textillate({

  loop: true,

  sync: true,

  in: {

    effect: "fadeInUp",

    sync: true,

  },

  out: {

    effect: "fadeOutUp",

    sync: true,

  },

});

// mic button click event

$("#micbutton").click(function(){

  if (i==0){

    eel.play_sound()

    i+=1

  }

  $("#Oval-Section").attr("hidden", true);

```

```
$("#SiriWave-Section").attr("hidden", false);
```

```
eel.allCommands()
```

```
})
```

```
});
```

command.py

```
import eel
```

```
import pyttsx3
```

```
from playsound import playsound
```

```
import speech_recognition as sr
```

```
import webbrowser
```

```
import time
```

```
import pyautogui
```

```
import os
```

```
import subprocess
```

```
#Speak function using pyttsx3
```

```
def speak(text):
```

```
    engine = pyttsx3.init()
```

```
    voices = engine.getProperty('voices')
```

```
    engine.setProperty('voice', voices[1].id)
```

```
    engine.setProperty('rate', 165) # Lower values make speech slower
```

```
engine.say(text)
```

```
engine.runAndWait()
```

```
#speak function using uk accent
```

```
# speak command
```

```
# def speak(command):
```

```
#     gtts = gTTS(command, lang='en', tld='com.in')
```

```
#     gtts.save('hello1.mp3')
```

```
#     playsound('hello1.mp3')
```

```
#speech_recognizing
```

```
def takecommand():
```

```
    r=sr.Recognizer()
```

```
    with sr.Microphone() as Source:
```

```
        speak("Hello, I am coral....")
```

```
        print("listening....")
```

```
        eel.DisplayMessage("Listening....")
```

```
        r.pause_threshold=1
```

```
        r.adjust_for_ambient_noise(Source)
```

```
audio=r.listen(Source,10,6)
```

```
try:
```

```
    print("Recogining...")
```

```
    eel.DisplayMessage("Recogizing....")
```

```
    query=r.recognize_google(audio,language="en-in")
```

```
    print(f"user said:{query}")
```

```
    eel.DisplayMessage(query)
```

```
except Exception as e:
```

```
    return query.lower()
```

```
return query.lower()
```

```
@eel.expose
```

```
def allCommands():
```

```
    query1=takecommand()
```

```
    print(query1)
```

```
    if "open youtube" in query1:
```

```
        eel.DisplayMessage("Here you go to Youtube")
```

```
        speak("Here you go to Youtube\n")
```

```
        webbrowser.open("youtube.com")
```

```
    elif "google" in query1:
```

```
        eel.DisplayMessage("Here you go to Google")
```



```

speak("Here you go to Google\n")

webbrowser.open("google.com")

elif "open notepad" in query1:

    speak("here u go to notepad...")

    subprocess.Popen("notepad.exe")


elif "screenshot" in query1:

    speak("ok takeing screenshot")

    im = pyautogui.screenshot()

    speak("imagename")

    saveimg="screenshot"

    extenion1=".jpg"

    im.save(saveimg+extenion1)

    speak("successfully taken scereenshot")

elif "open microsoft word" in query1:

    speak("here u go to microsoft word...")

    wordPath = "C:\\ProgramData\\Microsoft\\Windows\\Start
Menu\\Programs\\Word.lnk"

    os.startfile(wordPath)


elif "open calculator" in query1:

    speak("here u go to calculator...")

```

```
subprocess.Popen("calc.exe")
```

```
elif "open clock" in query1:
```

```
    speak("opening clock...")
```

```
    subprocess.Popen("clock.exe")
```

```
else:
```

```
    from engine.features import chatBot
```

```
    chat1=chatBot(query1)
```

```
    chat1=str(chat1)
```

```
    eel.DisplayMessage(chat1)
```

```
    print(chat1)
```

```
    speak(chat1)
```

```
    time.sleep(8)
```

```
    eel.showhood()
```

features.py

```
from playsound import playsound
```

```
import eel
```

```
from hugchat import hugchat
```

```
import webbrowser
```

```
import pyttsx3
```

```
import os
```

```

from engine.config import ASSISTANT_NAME

from engine.command import speak


#starting sound of EDITH


@eel.expose

def play_sound():

    sound_file="www\\bot_files\\audio_file\\www_assets_audio_start_sound (1).mp3"

    playsound(sound_file)


play_sound()


# play_sound


def speak1(text):

    engine = pyttsx3.init()

    voices = engine.getProperty('voices')

    engine.setProperty('voice', voices[1].id)

    engine.setProperty('rate', 165) # Lower values make speech slower

    engine.say(text)

    engine.runAndWait()

```

```
def openCommand(query):

    query = query.replace(ASSISTANT_NAME, "")

    query = query.replace("open", "")

    query.lower()

    if query!="":

        speak("opening"+query)

        os.system("start"+query)

    else:

        speak1("please say the name of the app")


def chatBot(query):

    user_input = query.lower()

    chatbot = hugchat.ChatBot(cookie_path="engine\cookies.json")

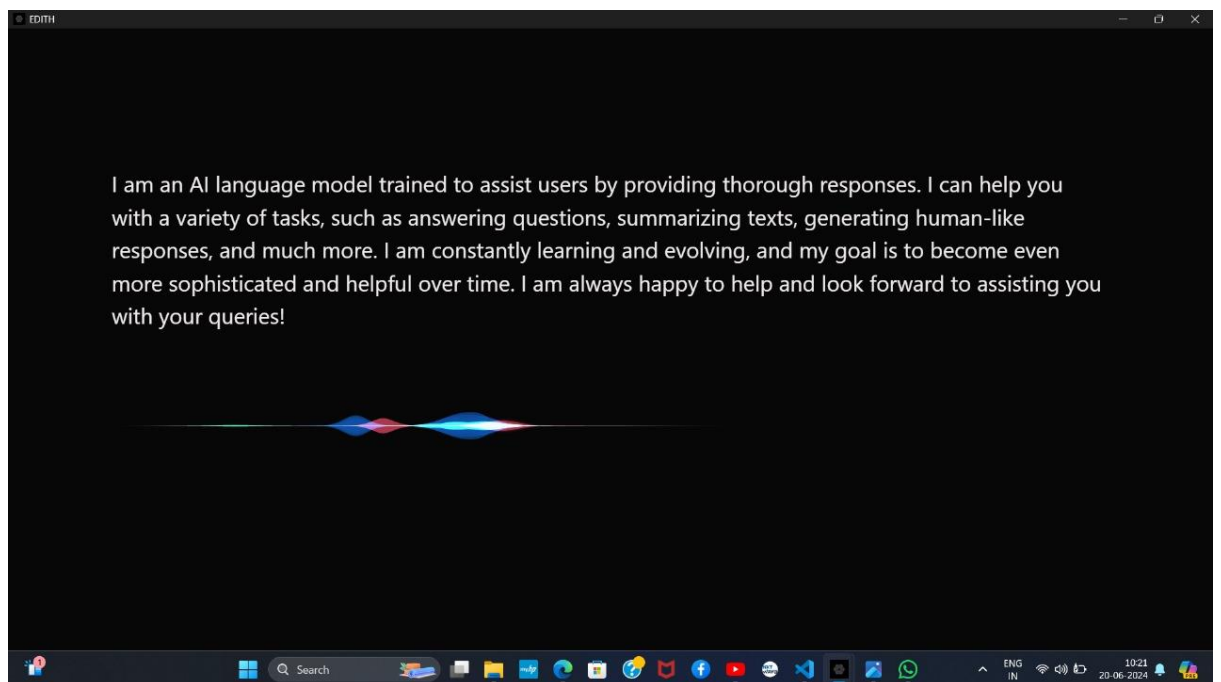
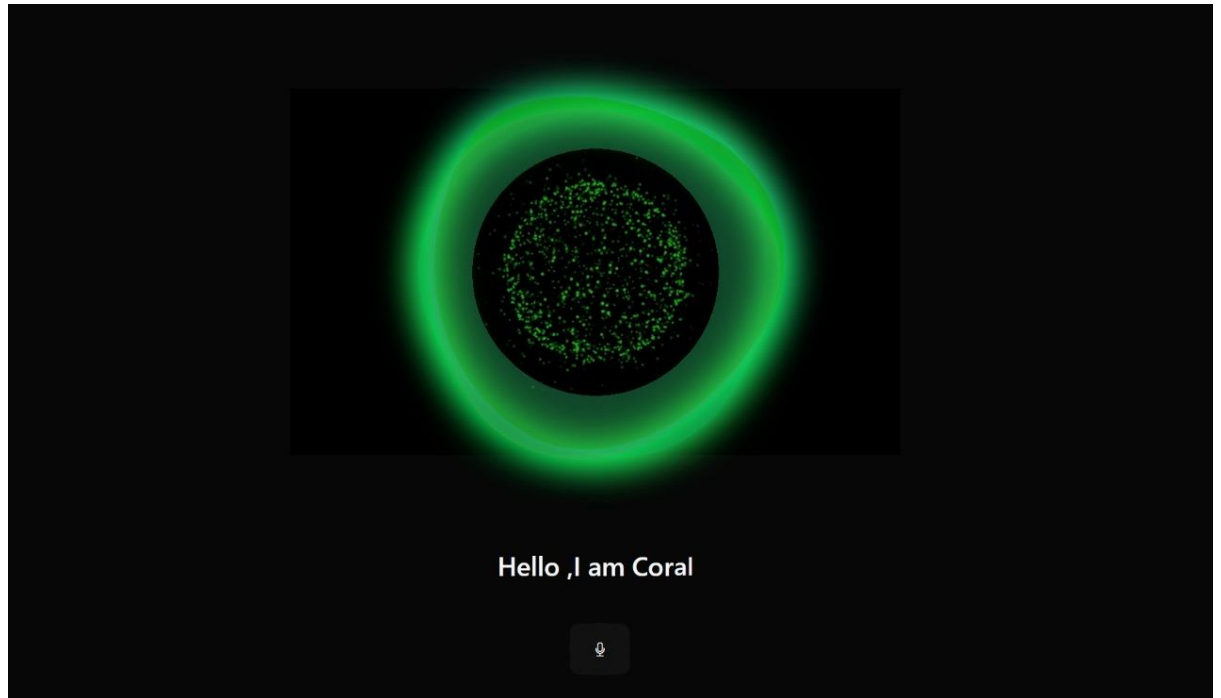
    id = chatbot.new_conversation()

    chatbot.change_conversation(id)

    response = chatbot.chat(user_input+"in 500 characters")

    return response
```

5.3 SCREENSHOTS

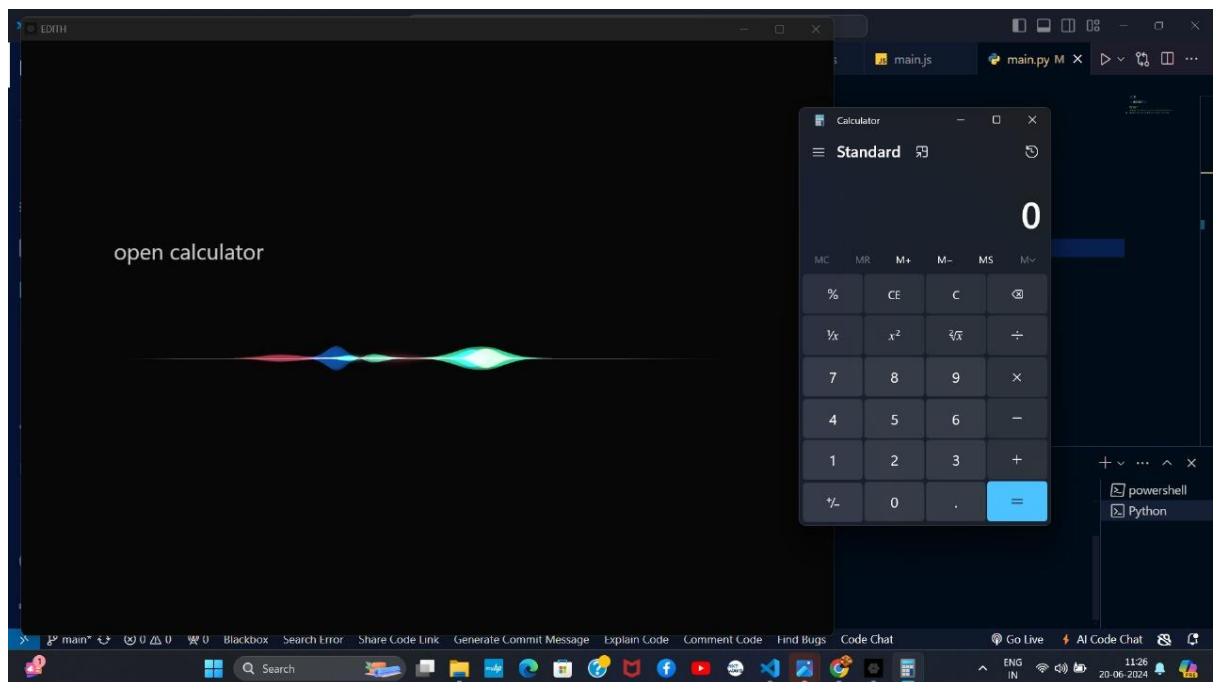
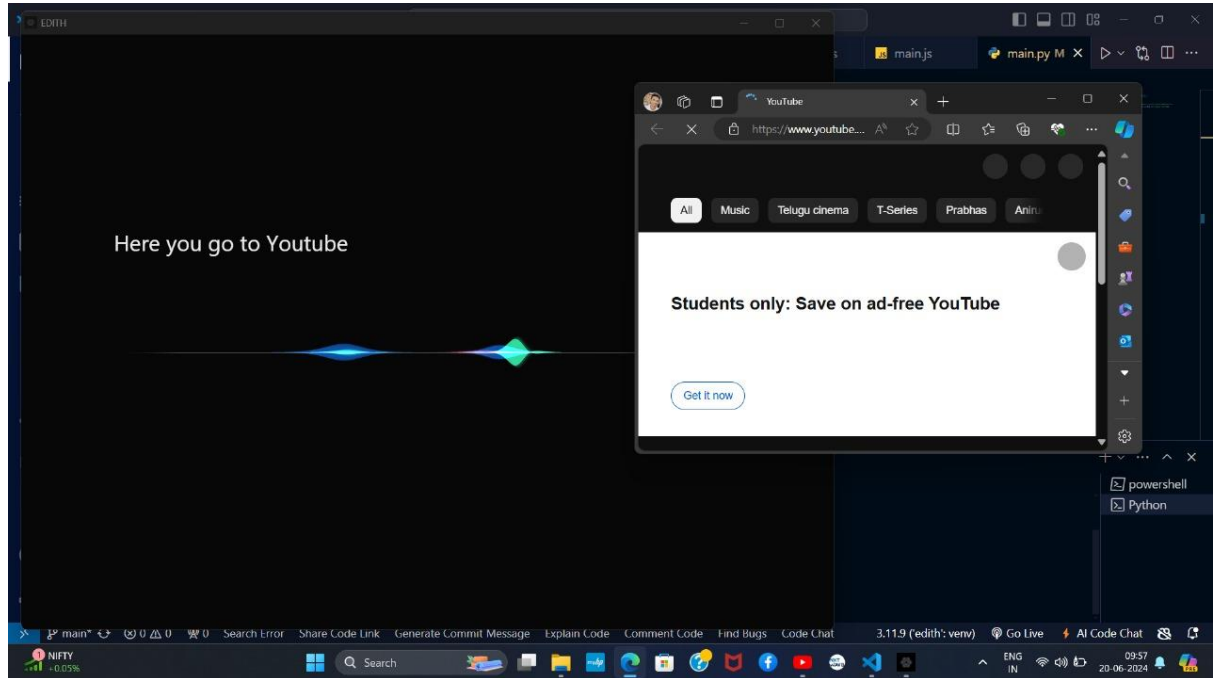


what is mean by design and analysis of algorithm in computer science

Design and Analysis of Algorithms is a field in computer science that focuses on creating and evaluating algorithms to solve computational problems efficiently. It involves designing algorithms with clear instructions for executing a task and analyzing their performance in terms of time and space complexity. This field ensures that algorithms are correct, efficient, and optimized for a given problem, contributing to the development of efficient software systems.

Tell me about Virat kohli

Virat Kohli is a star batter for the Indian cricket team. He has been the highest run-scorer in the last two World Cups. However, during a recent unexpected dip in form during the 2024 T20 World Cup, Kohli scored only 5 runs in three innings in India's first three matches, which may put his place in the team at risk. It is thought that the pitches in the USA did not suit his batting style, and he is expected to regain his form during the Caribbean leg of the tournament.



CHAPTER-6

TESTING and VALIDATION

- Software testing is a critical element of software quality assurance and represents the ultimate reviews of specifications, design and coding. Testing represents interesting anomaly for the software. During earlier definition and development phases, it was attempted to build software from an abstract concept to tangible implementation.
- The testing phase involves the testing of the developed system using various test data. Preparation of the test data plays a vital role in the system testing. After preparing of the test data the system under study was tested using those test data .while testing the system, errors were found and corrected by using the following testing steps and corrections are also noted for future use. Thus, a series of testing is performed for the proposed system, before the system was ready for the implementation.
- **Unit Testing**

Unit testing focuses verification effort on the similar unit of software design the form this is known as testing, the is done individually on each form. Using the unit test plans, prepared in design phase of the system development as a guide, important control paths are tested to uncover error within the boundary of the module. In this testing step, each module is found to be working satisfactorily, as regard to expected output from the module.

➤ **Integrated Testing**

Data can be lost across an interface, one module can have an adverse effect on other sub functions, when combined, may not produce the desired major function. Integration testing a systematic technique for constructing the program structure while at the same time conducting tests to uncover errors associated with the interface .all modules are combined in the testing step. Then the entire program is tested as a whole.

➤ **Output Testing**

After performing validation testing, the next steps are output testing of the proposed system, since no system could be useful if it does not produce the desired output in the specified format. The output generated is display by the system under in the specified format. The outputs generated are displayed by the system under consideration or tested by asking the user about the format required by them.

CHAPTER-7

CONCLUSION and FUTURE ENHANCEMENT

7.1 CONCLUSION

In this Paper we have discussed uses, methodology as well as implementation details of the personal Desktop based voice assistant using Python which is built using open-source software VS CODE as an implementation tool. This Project will be helpful for people of all generations as well as to people with some disabilities or people with some special cases. The personal voice assistant will be easy to use and will reduce the manual human efforts for performing various tasks. The functionality of the current voice assistant system is limited to working on Desktop based and working online (required to have internet connection to perform tasks) only. The voice assistant system is modular in nature so that addition of new features is possible without disturbing current system functionalities.

7.2 FUTURE ENHANCEMENT

- ✓ Make the virtual assistant learn more on its own and develop new skills with it.
- ✓ A virtual assistant Android application can also be developed.
- ✓ Increase the number of voice terminals available.
- ✓ Voice commands can be encrypted to maintain security.

The following are some of the places that may be relevant for the implementation of virtual assistants in the future:

a. Organisational Inquiry Desk: The system may be utilised in different organisations for simple access to information about the organisation using voice commands.

b. Embedded Systems: In embedded systems, voice commands may be used to handle multiple activities using speech recognition technology. This promotes the automation of labour and can thus be very advantageous in industrial process automation.

c. Application for People with Disabilities: People with disabilities may also benefit from voice recognition software. It is particularly beneficial for those who are unable to use their hands.

REFERENCES

<https://github.com>

<https://chatgpt.com>

<https://huggingface.co>